Step 2 — One Tool, One Loop: the Smallest Possible Working Agent — Hints

Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — One tool, and what the model sends back
- Hint 2 — The loop, and almost the whole thing

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — One tool, and what the model sends back

A tool is a normal function with `@tool` on top (Doc06). The docstring is what the model reads to decide when to use it — write it like a prompt: what it does, and when to use it.

`model.bind_tools(ALL_TOOLS)` gives you a model that *may* ask for a tool. Its reply is either:

- plain text in `response.content`, with `response.tool_calls` empty — the answer; or
- one or more requests in `response.tool_calls` — each a dict with `"name"`, `"args"` and `"id"`.

A tool request is not an answer. You run the tool and hand the result back — then ask again.

Build the Worker in its **finished** shape now, with one tool in the list — Step 3 will only add tools to it.

- `ALL_TOOLS = [celsius_to_fahrenheit]`
- `find_tool(tools, name)` — loop over the list, compare `.name`, return the tool or `None`.
- `run_tool(call, tools)` — `(None, "unknown tool: ...")` if not found; otherwise `try: tool.invoke(call["args"])` → `(str(result), None)`, and on **any** exception → `(None, str(e))`.

`tool.invoke(...)` checks the arguments with Pydantic first (from the type hint `celsius: float`), so `{"celsius": "hot"}` becomes an error observation, not a crash.

> Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The loop, and almost the whole thing

```
run_worker(model, task, max_iterations):
    model_with_tools = model.bind_tools(ALL_TOOLS)
    messages = [system from worker.txt, user task]
    steps = []
    for step in range(max_iterations):
        response = model_with_tools.invoke(messages)
        messages.append(response)
        if not response.tool_calls:
            return AgentResult(response.content, steps)
        for call in response.tool_calls:
            result, error = run_tool(call, ALL_TOOLS)
            append {"role": "tool", "tool_call_id": call["id"],
                    "content": result or "Error: " + error}
            steps.append({step, tool, arguments, result, error})
    raise MaxIterationsExceeded("No answer after N steps", steps)
```

`AgentResult` is a `@dataclass` with `final_answer` and `steps: list[dict] = field(default_factory=list)` (Doc07). `MaxIterationsExceeded(message, steps)` goes in `exceptions.py` and stores `steps`, so the caller still sees what happened.

In `main.py`: `MAX_ITERATIONS = 5`, a `print_steps()` helper, and `except MaxIterationsExceeded` (print the partial log, go on) plus `except openai.RateLimitError`.

Tests:

Every case prints `OK` or `WRONG` (the Step 1 pattern), one function per case:

- `test_tools.py` — `celsius_to_fahrenheit.invoke({"celsius": 18})` → `64.4`; `{"celsius": "hot"}` → `ValidationError`.
- `test_agents.py` — `find_tool()` with a known and an unknown name, and `run_tool()` with hand-made call dicts: a good call, an unknown tool, bad arguments (5 cases).
- `test_live_agents.py` — a conversion uses the tool, a general question uses none, `max_iterations=1` raises.

Full solution: Show me the solution